AI 协作日志 W9

文件命名：[王梓健][2024150200]AI 协作日志 W9.docx

姓名：王梓健

学号：2024150200

使用的 AI 工具及方式：DeepSeek（AI 编程助手）。先用结构化 Prompt 把业务契约与 6 个测试维度写清楚，让它生成 TypeScript 骨架和 Vitest 用例；再由我逐条审查断言是否“改错了也会红”、对照 9 维清单补边界，并亲手改写弱断言与漏测用例。

Git 仓库链接：https://github.com/kartwatt-cpu/web-week9（公开仓库，含 feature/typescript-refactor 分支与合并记录）

任务：完成 V2.0 的 TypeScript 重构与单元测试。包括：

1. 将 W8 的 JS 代码重构为 TS（处理 null 与脏数据，严禁 any）。
2. 手写单测（覆盖正常与边界）。
3. 使用结构化 Prompt 引导 AI 生成测试代码，并进行审查。

一、任务 1：TypeScript 重构（两句话记录）

任务要求：加上类型约束（interface Todo）；处理 dataset.id 字符串到 number 的转换；消除 null 报错，严禁 any。完整代码在 Git 仓库中验收。

我让 AI 做了：把 W8 页面里内联的 Todo 脚本抽成 todo.ts（数据层）和 app.ts（视图层），定义 interface Todo 与 Filter 字面量联合类型，给 addTodo / toggleTodo / removeTodo / editTodo / parseTodoId 补全参数与返回值类型，并把 e.target 先收窄成 Element、再用 closest<HTMLLIElement>('li') 取任务，最后用 Number() + Number.isSafeInteger 把 dataset.id 转成 number；在 strict: true + noEmit: true 下把隐式 any 与 null 报错清零。

我做了：逐行审查并动手改了三处——①把 addTodo 的 text 参数从 string 改成 unknown，强制函数内部先做 typeof / trim / 长度 / 标签清洗的运行时校验，因为 input.value 与外部调用都可能塞进 null；②把 id 校验抽成 parseTodoId(raw: string | undefined): number | null，先 trim 再用 /^\d+$/ 挡住 '1.5'、'123abc'，最后用 Number.isSafeInteger 兜底；③删掉 AI 顺手留下的 as any 与非空断言，改成 instanceof Element 收窄 + null 判断，tsc --noEmit 复查为零报错，并用 headless 浏览器实际点了切换 / 删除 / 空白处 / 脏 id 四种场景。

二、任务 2：手写单测实战

任务要求：不依赖 AI，为 addTodo 和 toggleTodo 手写基础单测（覆盖正常情况与边界拦截）。

1. 我手写的测试代码（核心片段）

```
import { test, expect } from 'vitest';
import { addTodo, toggleTodo } from './todo';
import type { Todo } from './todo';
test('addTodo：正常添加，长度 +1 且文本正确', () => {
  const todos: Todo[] = [];
  addTodo(todos, '学习 TS');
  expect(todos.length).toBe(1);
  expect(todos[0].text).toBe('学习 TS');
});
test('addTodo：纯空格被拦截', () => {
  const todos: Todo[] = [];
  addTodo(todos, '   ');
  expect(todos.length).toBe(0);
});
test('addTodo：空字符串被拦截', () => {
  const todos: Todo[] = [];
  addTodo(todos, '');
  expect(todos.length).toBe(0);
});
test('toggleTodo：正常切换，且不影响其他任务', () => {
  const todos: Todo[] = [
    { id: 1, text: '学习 TS', completed: false },
    { id: 2, text: '写单测', completed: false },
```

```
  ];

  toggleTodo(todos, 1);

  expect(todos[0].completed).toBe(true);

  expect(todos[1].completed).toBe(false);

});

test('toggleTodo：不存在的 ID 不崩溃', () => {

  const todos: Todo[] = [];

  toggleTodo(todos, 999);

  expect(todos.length).toBe(0);

});
```

2. 运行截图

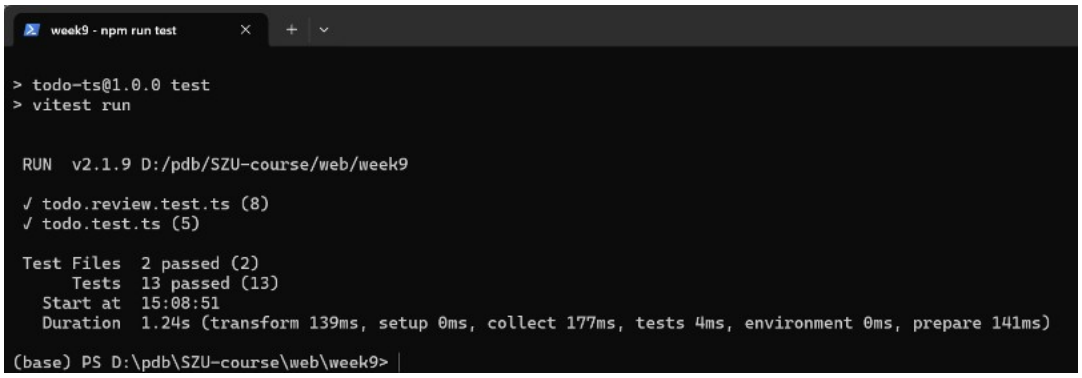


图 1 npm run test 真实终端输出：todo.test.ts 的 5 条手写用例全部通过，todo.review.test.ts 的 8 条修补用例也通过，共 Test Files 2 passed / Tests 13 passed

3. 手写测试时踩的坑（选填）

第一个坑是“长度断言看起来够了”：我一开始只写 expect(todos.length).toBe(1)，故意把 addTodo 里的 text 改成空串，测试居然还是绿的——长度只能证明“加进去了”，证明不了“加对了”，补上 expect(todos[0].text).toBe('学习 TS') 才抓得住。第二个坑是 toggleTodo 只测了目标翻转，漏了“其他任务不受影响”，补第二条断言时才发现自己根本没验证“只动一条”。第三个坑是把 todos 写在 describe 外层，单独跑第二条时被第一条的数据污染，改成每个 test 内部新建 Todo[] 才稳定。

三、任务 3：AI 生成测试与审查

任务要求：使用结构化 Prompt 让 AI 生成测试，并按审查原则挑出 AI 的毛病，进行人工修补。

1. 我的 Prompt（提问约束）

我有一段 TypeScript 代码 todo.ts，业务契约如下：

1. 合法文本（trim 后非空、长度 <= 200）才添加任务；id 唯一，completed 初始为 false。

2. 纯空格、空字符串、null、undefined、非字符串一律拒绝，不改变 todos。

3. 含 HTML 标签的文本要在数据层清洗，<script> 不能原样存入。

4. toggleTodo 按 id 翻转 completed；id 不存在时安静返回，不抛错、不改动数组。

请使用 Vitest 编写测试，并严格按以下维度覆盖：

- 维度 1（空值）：null、undefined。

- 维度 2（格式）：空字符串、纯空格、201 字超长文本。

- 维度 3（类型不匹配）：数字、对象等错误类型。

- 维度 4（越界/不存在）：不存在的 id，且已有数据不能被改动。

- 维度 6（重复）：连续添加相同文本。

- 维度 9（安全）：<script>alert(1)</script> 注入。

断言要求：必须验证具体业务结果（长度、text、completed），不能只写 toBeDefined / toBeTruthy。

请在每个 test 上方用注释标明对应维度；测试数据必须写在每个 test 内部。

2. AI 生成的测试代码（核心片段）

```
import { test, expect } from 'vitest';

import { addTodo, toggleTodo } from './todo';

import type { Todo } from './todo';

const todos: Todo[] = [];    // 写在文件顶层，前两个用例共享

// 正常路径：添加一条任务

test('添加一条任务', () => {
```

```
  addTodo(todos, '学习 TS');
  expect(todos[0]).toBeDefined();
});
// 维度 2（格式）：空字符串
test('拒绝空字符串', () => {
  addTodo(todos, '');
  expect(todos.length).toBe(1);
});
// 维度 2（格式）：纯空格
test('拒绝纯空格', () => {
  const todos: Todo[] = [];
  addTodo(todos, '   ');
  expect(todos.length).toBe(0);
});
// 维度 6（重复）：连续添加相同文本
test('重复添加相同文本', () => {
  const todos: Todo[] = [];
  addTodo(todos, '学习 TS');
  addTodo(todos, '学习 TS');
  expect(todos.length).toBe(2);
});
// 维度 4（越界）：不存在的 ID
test('不存在的 ID 不崩溃', () => {
  const todos: Todo[] = [];
  toggleTodo(todos, 999);
  expect(todos.length).toBe(0);
});
```

3. 审查报告（必填）

① 有效性审查：找出 AI 生成测试代码的问题：

AI 写的测试里，哪一句/哪一些断言是无效的？为什么？

无效断言有三处。

① 第 1 个 test（添加一条任务）里的 expect(todos[0]).toBeDefined()：它只证明数组第 0 项存在，证明不了文本内容。把 addTodo 改成 todos.push({ id: nextId(), text: '', completed: false })，这条断言依然通过——这正是“实现改错了也不变红”的弱断言，必须改成 expect(todos[0].text).toBe('学习 TS')。

② 第 2 个 test（拒绝空字符串）里的 expect(todos.length).toBe(1)：契约写的是“空字符串必须拒绝”，期望值应是 0。它把“没拦住”写成了预期，属于期望值写反。

③ 顶层 const todos: Todo[] = [] 被前两个用例共享：第一个用例已经加了一条，第二个用例即使真被拦截，长度也还是 1，toBe(1) 刚好把污染掩盖成“通过”；单独跑第二个用例结果就会变，这就是测试污染。

② 边界审查：对照 9 维度查漏补缺

对照 9 大维度（空值/格式/类型/越界/状态/重复/异步/依赖/安全），AI 遗漏了哪个当前业务需要的边界？

对照 9 维，AI 漏了 5 项：维度 1 的 null / undefined、维度 2 的 201 字超长文本、维度 3 的类型不匹配（数字 / 对象）、维度 4 的“不存在的 id 不能改动已有数据”、维度 9 的 <script> 注入。当前业务最需要先补的是维度 1 和维度 9：

① input.value 在运行时可能被外部调用替换成 null，AI 的 5 条用例一条都没测；而 addTodo 里一旦直接 text.trim() 就会抛 TypeError，所以 null 必须在函数入口被挡住。

② 渲染层虽然用 textContent，但数据层契约要求 <script> 不能原样入库；AI 没测，等于把安全边界漏了。

另外维度 6 只断言了长度 2，没断言第二条的 text，补一条 expect(todos[1].text).toBe('学习 TS') 才能证明“重复添加的确实是同一条内容”。

③ 我的修补方案（手写或再次控制代码生成）

把 AI 漏掉的或写错的测试用例，手写修正或再次控制生成并补充在下面：

// ① 换掉弱断言

```
expect(todos[0].text).toBe('学习 TS');

// ② 维度 1：null / undefined 应被拦截

test('维度 1：null 被拦截', () => {
  const todos: Todo[] = [];
  addTodo(todos, null);
  expect(todos.length).toBe(0);
});

test('维度 1：undefined 被拦截', () => {
  const todos: Todo[] = [];
  addTodo(todos, undefined);
  expect(todos.length).toBe(0);
});

// ③ 维度 2：201 字超长文本

test('维度 2：201 字超长文本被拦截', () => {
  const todos: Todo[] = [];
  addTodo(todos, 'a'.repeat(201));
  expect(todos.length).toBe(0);
});

// ④ 维度 6：重复文本各自入库

test('维度 6：重复文本各自入库', () => {
  const todos: Todo[] = [];
  addTodo(todos, '学习 TS');
  addTodo(todos, '学习 TS');
  expect(todos.length).toBe(2);
  expect(todos[1].text).toBe('学习 TS');
});

// ⑤ 维度 9：<script> 在数据层被清洗
```

```
test('维度 9：script 标签不被原样存入', () => {
  const todos: Todo[] = [];
  addTodo(todos, '<script>alert(1)</script>');
  expect(todos[0].text).not.toContain('<script>');
  expect(todos[0].text).toBe('alert(1)');
});
// ⑥ 维度 4：不存在的 id 不改动已有任务
test('维度 4：不存在的 ID 不改动已有任务', () => {
  const todos: Todo[] = [{ id: 1, text: '学习 TS', completed: false }];
  toggleTodo(todos, 999);
  expect(todos.length).toBe(1);
  expect(todos[0].completed).toBe(false);
});
// ⑦ 维度 3：数字不是合法文本
test('维度 3：数字不是合法文本', () => {
  const todos: Todo[] = [];
  addTodo(todos, 123);
  expect(todos.length).toBe(0);
});
```

补完后运行 npm run test：Test Files 2 passed (2)，Tests 13 passed (13)，说明修补用例与手写的 5 条用例一起全绿。

四、一句话总结

最大的收获是：TS 只能保证“形状对”，测试才能验证“行为对”；而 AI 写的测试最容易“全绿但抓不到错”，必须用“把实现故意改坏一处，它会不会变红”去量，再拿 9 维清单补边界——现在我看到一条断言，会先问它到底证明了哪条业务结果，而不是看到绿色就放心。